

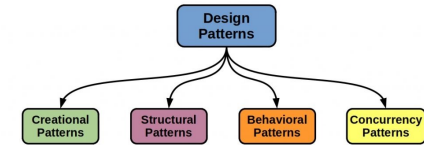
Ingeniería de Software II

Patrones de Diseño

Patrones de Comportamiento

Docente: Dr. Pedro E. Colla

Tipos de patrones



- **Patrones de creación**

Los patrones de creación abstraen la forma en la que se crean los objetos, permitiendo tratar las clases a crear de forma genérica dejando para más tarde la decisión de qué clases crear o cómo crearlas.

- **Patrones estructurales**

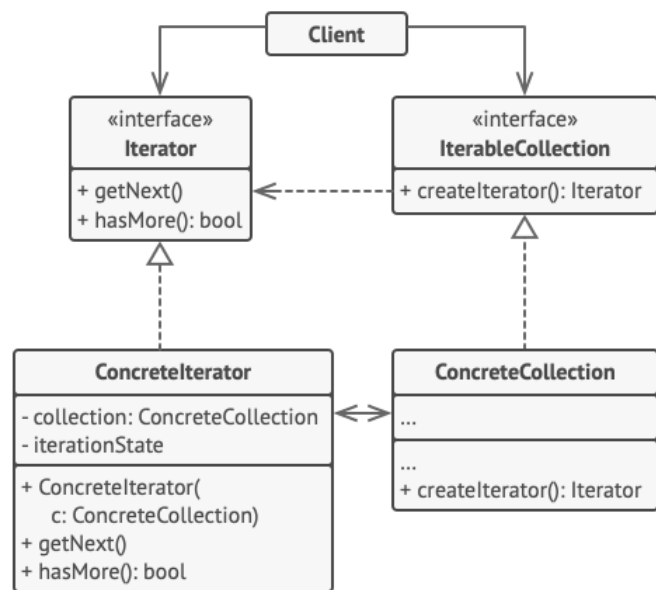
Tratan de conseguir que cambios en los requisitos de la aplicación no ocasionen cambios en las relaciones entre los objetos. Lo fundamental son las relaciones de uso entre los objetos, y, éstas están determinadas por las interfaces que soportan los objetos.

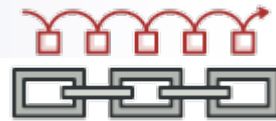
- **Patrones de comportamiento**

Los patrones de comportamiento estudian las relaciones de las llamadas entre los diferentes objetos, normalmente ligados con la dimensión temporal.

Creación	Estructurales	Comportamiento
Singleton Factory Builder Prototype	Adapter Bridge Composite Decorator Fachade Flyweight Proxy	Chain of responsibility Command Interpreter Iterator Mediator Memento Observer ... et al.

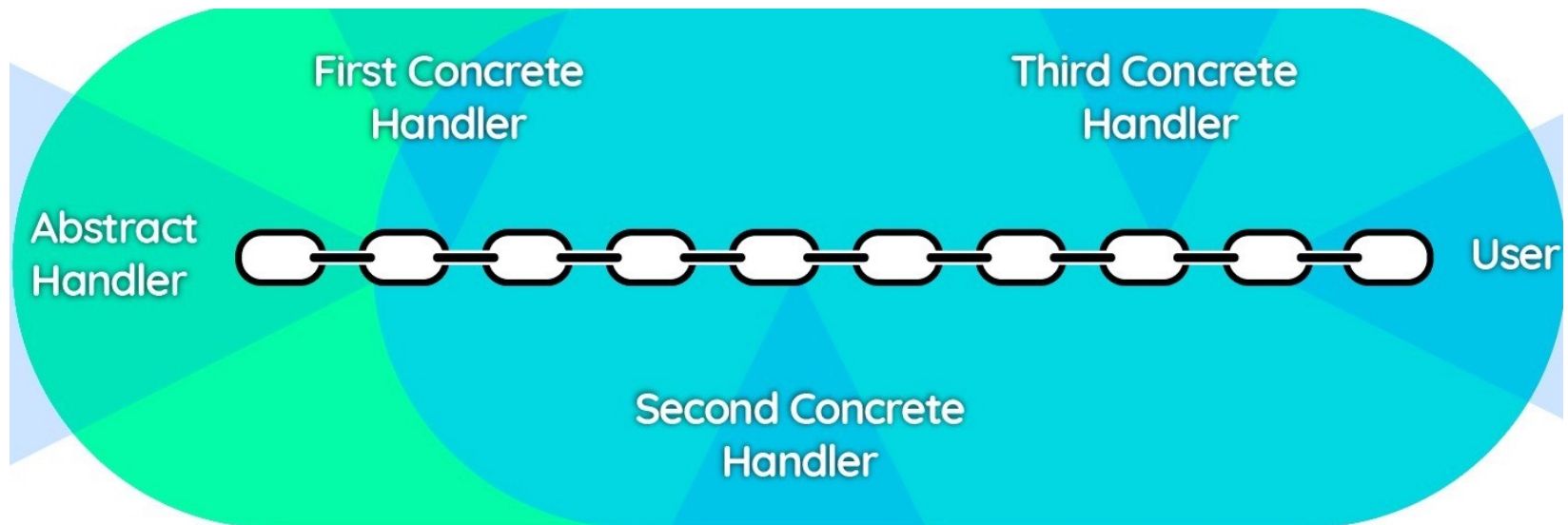
Patrones de comportamiento



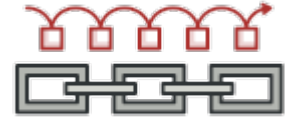


Chain of responsibility

Resuelve el problema de procesar en forma ordenada una serie de comandos, articulando un método para definir la secuencia y la resolución del siguiente comando



Chain of responsibility

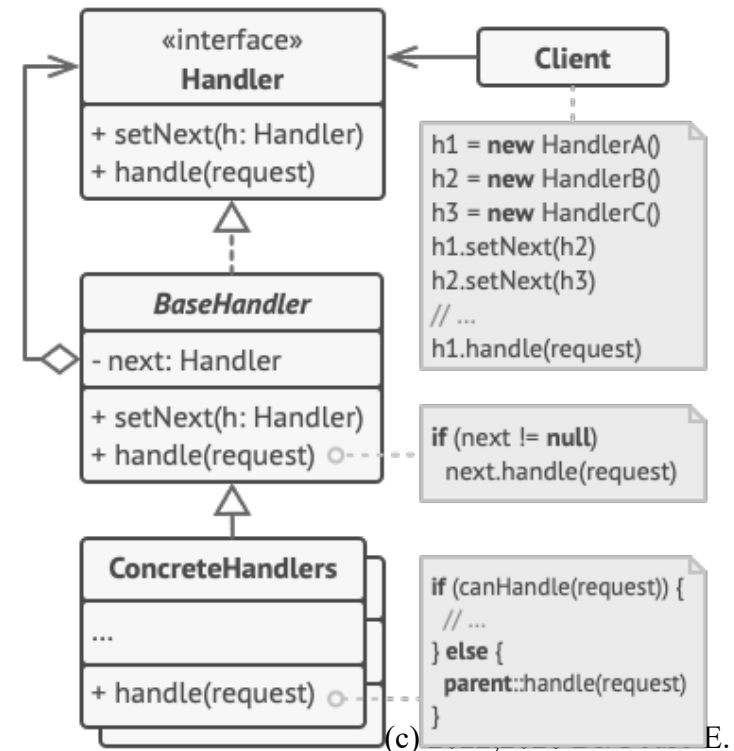


Chain of responsibility/Chain of command

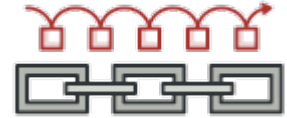
Es un patrón de comportamiento que permite pasar requerimientos entre una cadena de gestores. Cada uno al recibir el requerimiento decide si procesarlo, y consumirlo o dejarlo pasar al terminar.

Propósito	Problema
• Permite gestión ordenada de mensajes • Se puede introducir controladores adicionales sin cambiar la interfaz al cliente.	Mayor complejidad. Costo de construcción. Persistencia. Tiempo de procesamiento

python chain.py



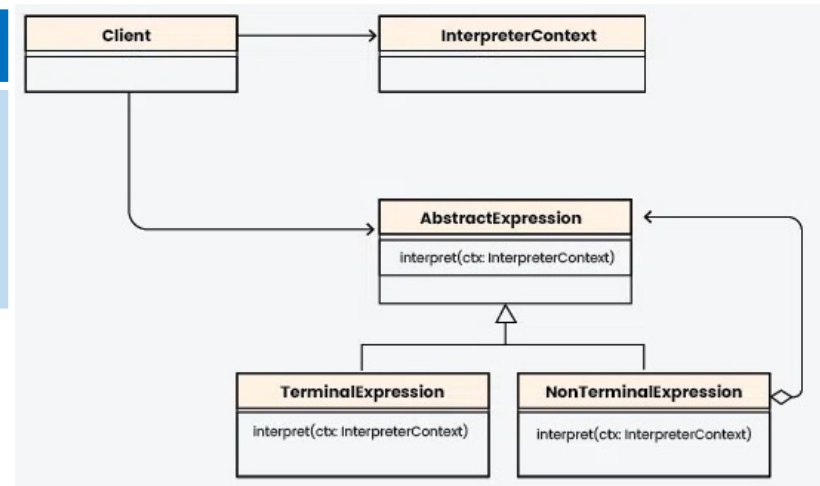
Interpreter



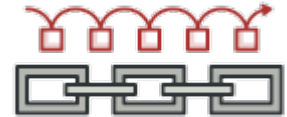
Interpreter

Es un patrón que permite implementar analizadores léxicográficos (parsers) y ejecutarlos en un contexto.

Propósito	Problema
Permite definir una sintaxis genérica y el parser para evaluarla.	Deben cuidarse las condiciones de excepción.



Iterator



Iterator

Es un patrón de comportamiento que permite recorrer una colección de objetos sin exponer su representación o implementación (lista, stack, árbol, matriz, etc.)

Propósito

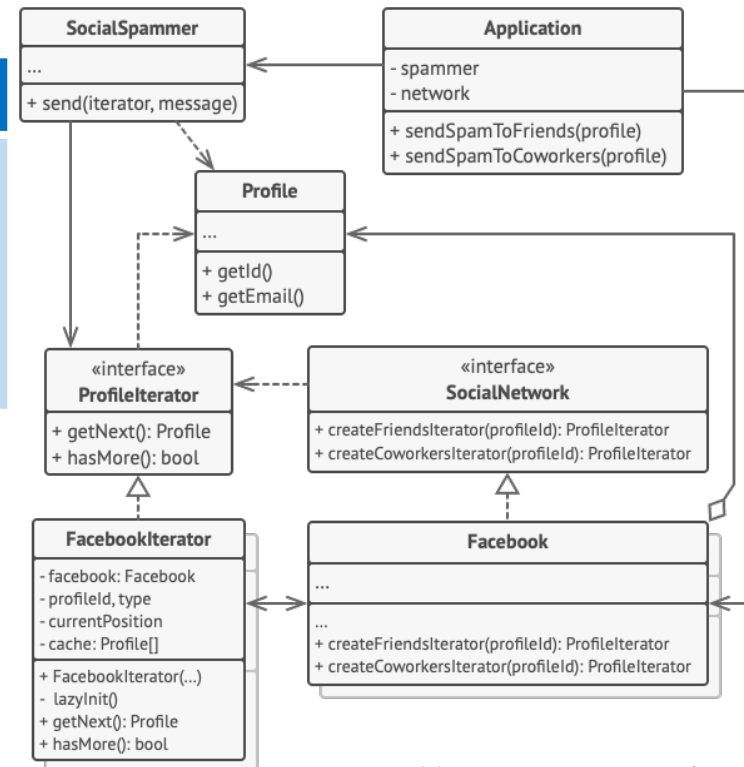
- Permite procesar todos los componentes.
- El código para iterar es genérico.

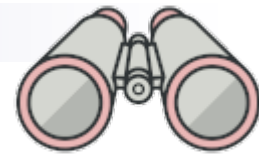
Problema

Mayor complejidad.
Menos eficiente que un iterador especializado.

En python su importancia es menor pues es un lenguaje que introduce el concepto de iterator en forma nativa para los objetos tipo colección.

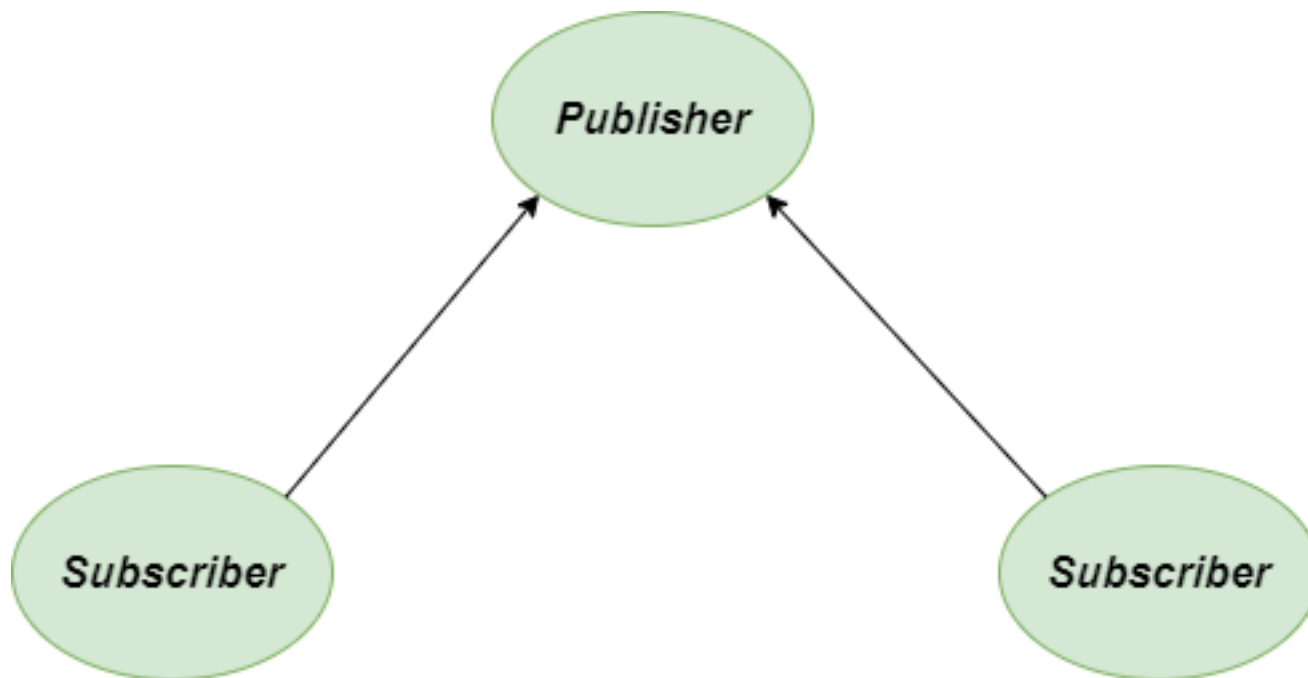
python iterator.py (iter1.py / iter2.py)

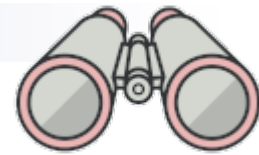




Observer

Resuelve el problema de lograr que objetos interesados (subscriber) en cambios sobre un determinado objeto (Publisher) reciban notificaciones sobre cambios de estado.



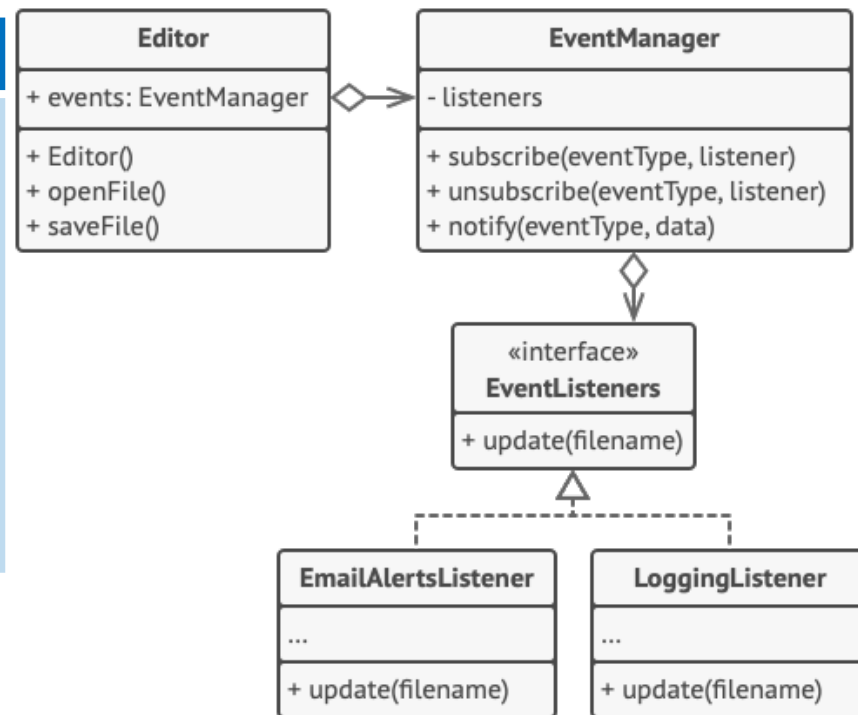


Observer

Observer

Es un patrón de comportamiento que permite definir un mecanismo de subscripción mediante el cual notificar a múltiples objetos sobre algún evento de interés.

Propósito	Problema
• Permite compartir eventos en forma flexible. • Implementa la transferencia transparente de información entre receptores no similares.	Mayor complejidad. Difícil establecer un orden de aviso a subscriptores

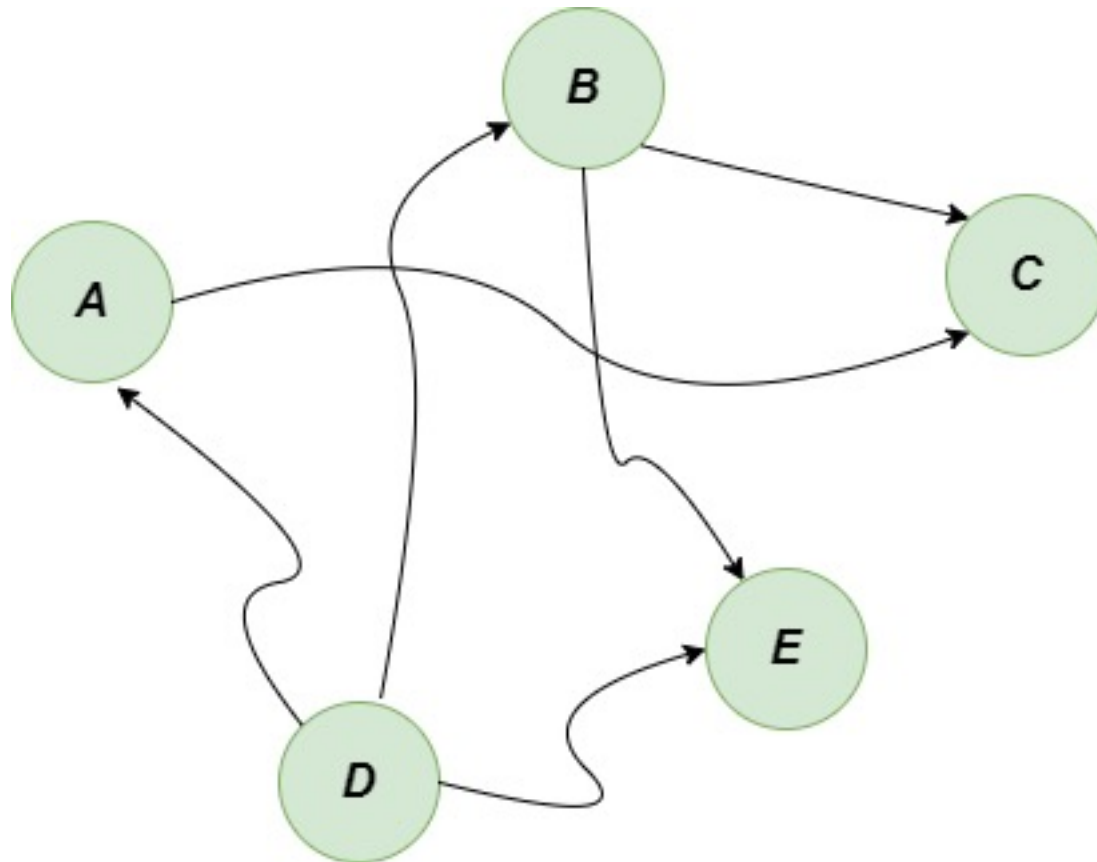


python observer.py

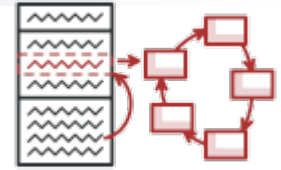
State



En múltiples ocasiones es necesario cambiar el estado de comportamiento de un objeto en función de la aplicación de eventos y reglas de procesamiento para los mismos.



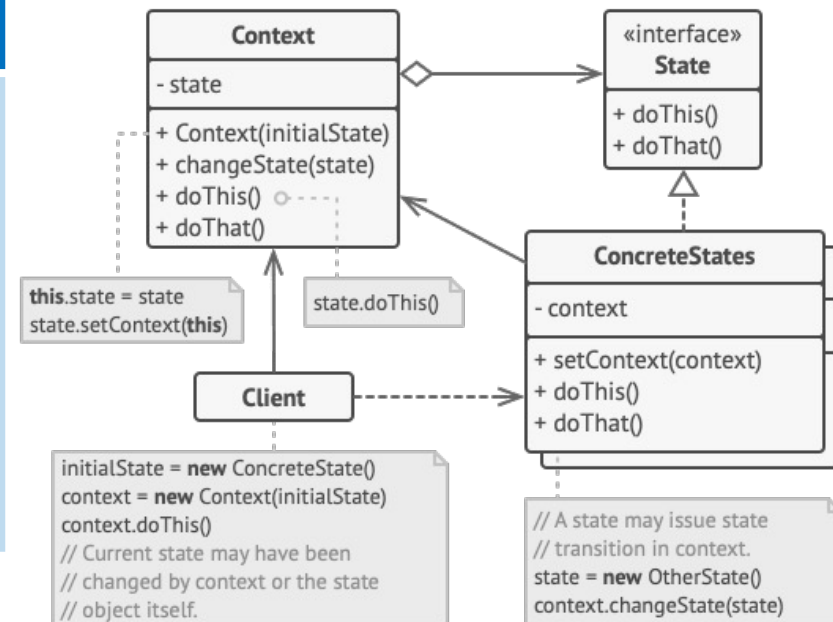
State



State

Es un patrón de comportamiento que permite que un objeto cambie su comportamiento de acuerdo a cierto estado interno, de tal manera que luzca como una clase distinta. Implementa autómatas de estado finito

Propósito	Problema
• Reduce acoplamiento. • Alterar la red de estados de una FSM sin modificar los existentes. • Reducir condicionales (McCabe)	Mayor complejidad. Puede ser excesivo para máquinas de estado simples.

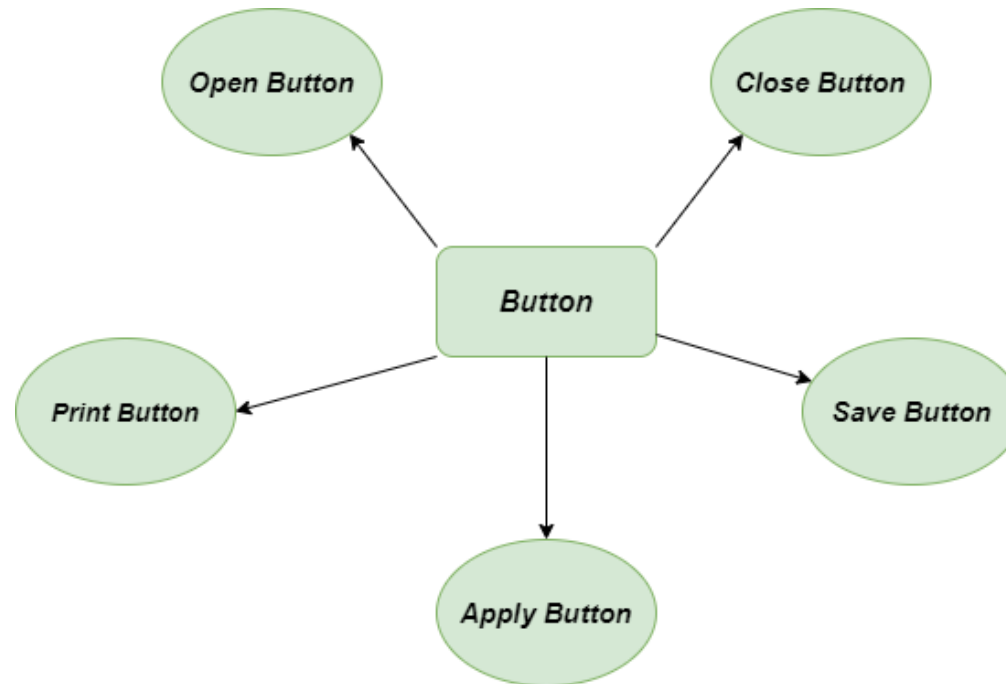


python state.py



Command

El problema a ser abordado es una situación se quiere evitar implementar múltiples sub-clases de comportamiento parecido.



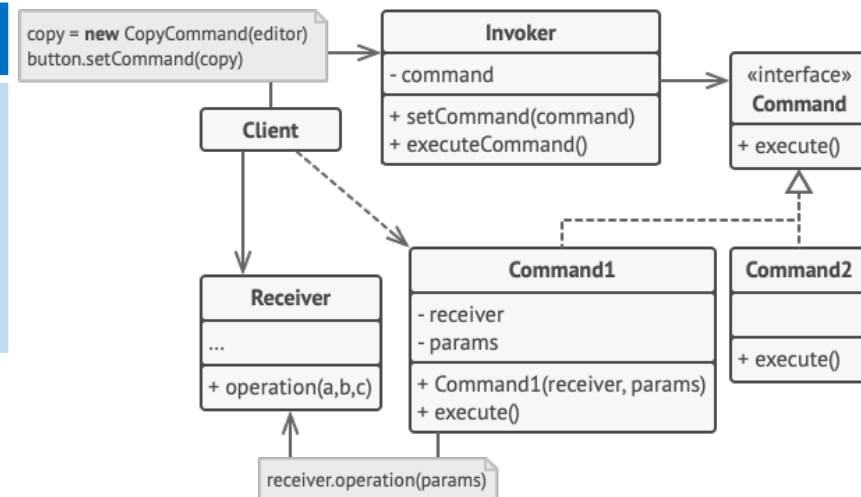


Command

Command

Es un patrón de comportamiento que permite convertir una solicitud en un objeto independiente que contiene toda la información sobre la solicitud.

Propósito	Problema
Gestionar la aplicación, retraso o priorización del comando junto a su contexto.	Mayor complejidad. Debe salvarse no solo el request, sino su método de ejecución y su contexto.



python command.py

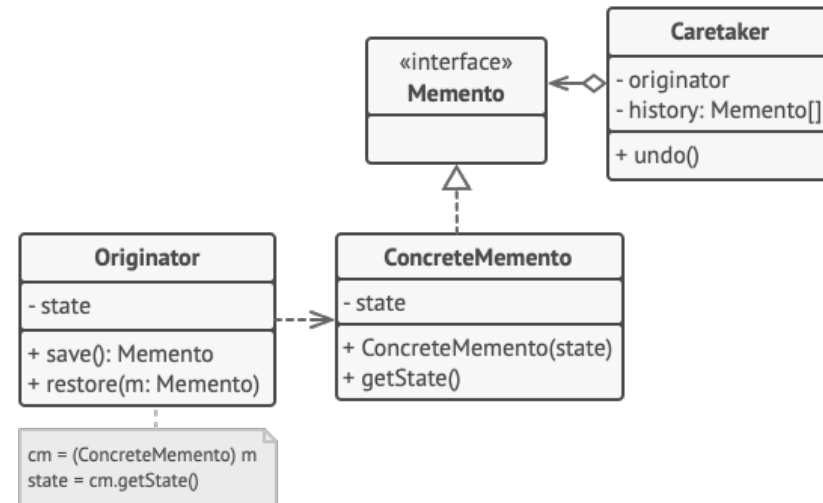
Memento



Memento

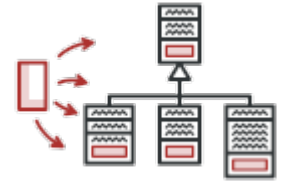
Es un patrón de comportamiento que permite almacenar información relevante sobre un objeto independientemente de su implementación y recuperar un estado anterior.

Propósito	Problema
• Permite guardar información sin revelar encapsulamiento o estructura (blob). • Gestionar historia de estados (snapshot, checkpoint).	Mayor complejidad. Garbage collection. Mayor consumo de recursos.



python memento.py

Visitor



Visitor

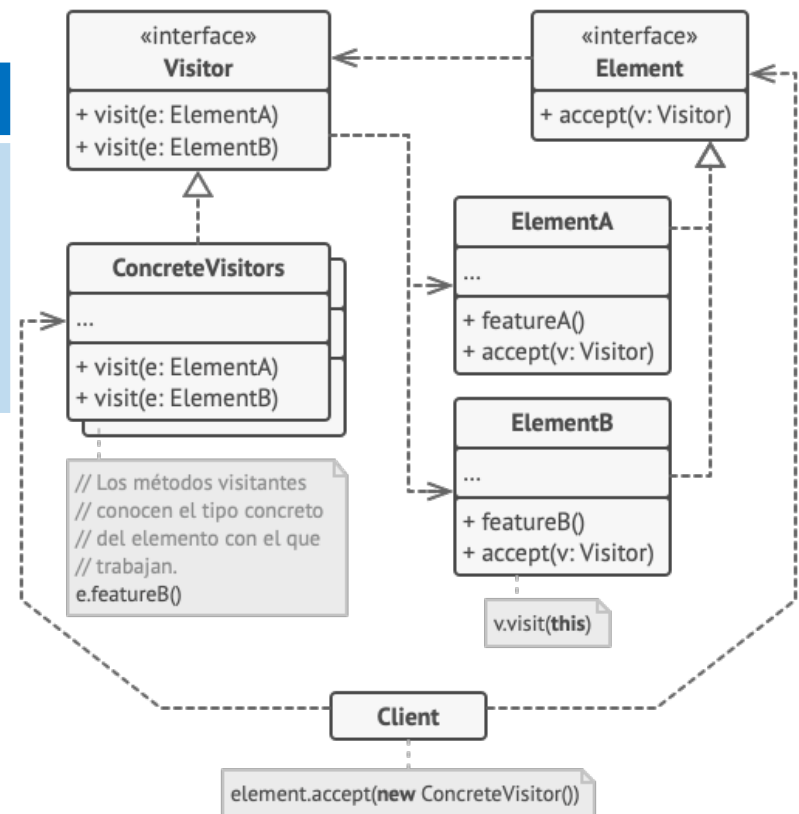
Es un patrón de comportamiento que permite separar algoritmos de los objetos sobre los que operan.

Propósito

Problema

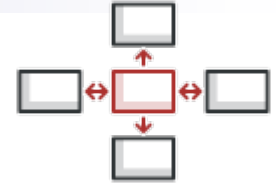
- Un único observante puede conocer información única sobre una jerarquía de objetos.

Mayor complejidad.



python visitor.py

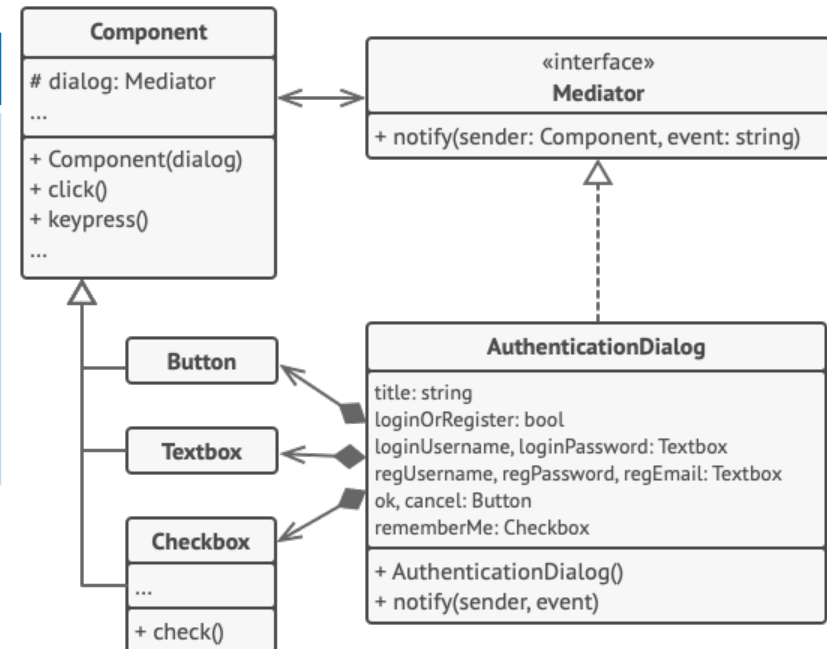
Mediator



Mediator

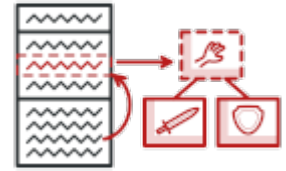
Es un patrón de comportamiento que gestiona la interacción entre objetos para impedir su intercambio directo y despachar requerimientos entre los mismos.

Propósito	Problema
• Reduce acoplamiento. • Permite diferir la definición de relaciones particulares entre objetos.	Mayor complejidad. Punto simple de falla. Gestión de estado y persistencia



python mediator.py

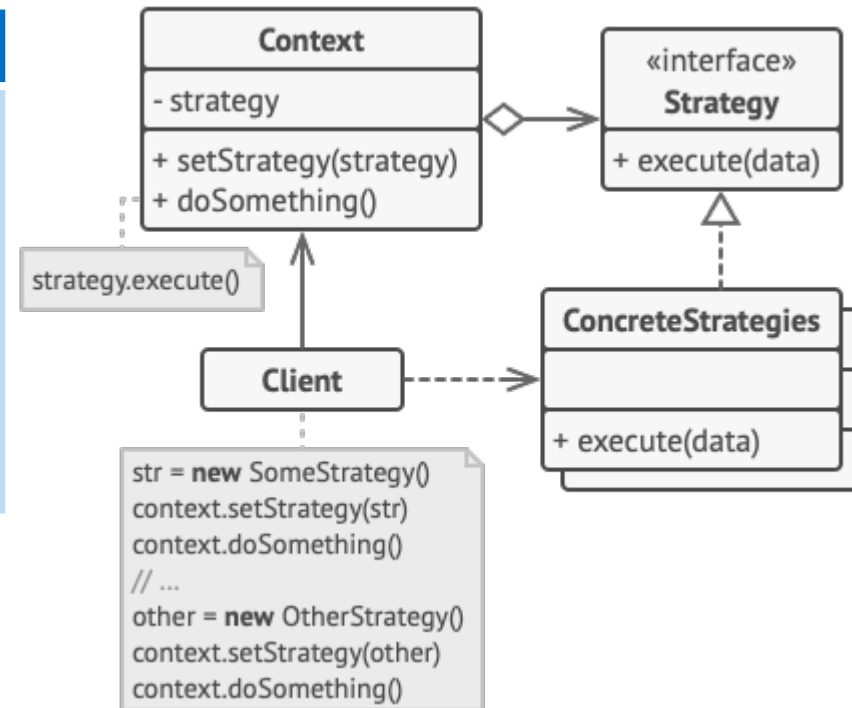
Strategy



Strategy

Es un patrón de comportamiento que permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables.

Propósito	Problema
- Selección dinámica de algoritmos en tiempo de ejecución. - Detalles del algoritmo diferentes de su utilización - Herencia por composición	Mayor complejidad. Determinar la forma mas eficiente de elegir la estrategia



python strategy.py



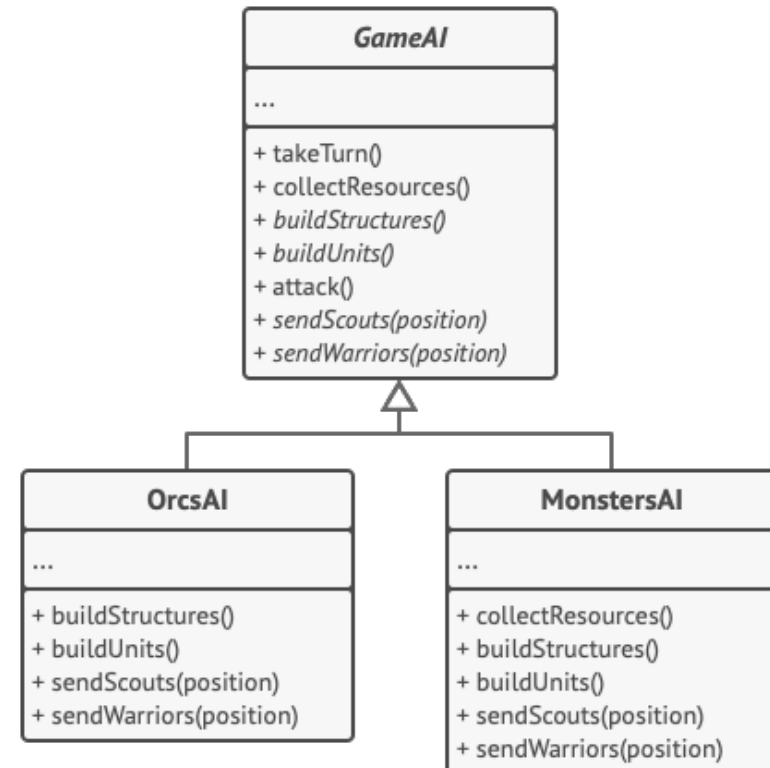
Template

Template

Es un patrón de comportamiento que define el esqueleto de un algoritmo en la superclase pero permite que las subclasses sobrescriban pasos del algoritmo sin cambiar su estructura..

Propósito	Problema
Permitir a los clientes que sobrescriban tan solo ciertas partes de un algoritmo grande, para que les afecten menos los cambios que tienen lugar en otras partes del algoritmo.	Mayor complejidad. Variante sofisticada de herencia.

python template.py



(c) 2022,2026 Dr. Pedro E.
Colla

Taller



Utilice los ejemplos de los distintos patrones de comportamiento

`ejemplo_*.py`

- Analice el código, verifique la forma de implementar.
- Ejercite variantes sobre los mismos.

¿Preguntas?



Bibliografía

- Architecture Patterns with Python – Percival & Gregory – O'Reilly
- Design Patterns – Lasater
- <https://refactoring.guru/design-patterns/python>
- <https://www.geeksforgeeks.org> (Python design patterns)